

STAR Proto Implementation Primer

Shared Protocol Buffer Definitions

STAR Development Team

December 2025

Contents

1 Project Overview

1.1 Purpose

The `star-proto` repository contains shared Protocol Buffer (protobuf) definitions used across the STAR robot system:

- **star-gateway**: Kotlin/Java gRPC server
- **star-ui**: TypeScript gRPC-Web client
- **star-esp32-firmware**: (Future) ESP32 binary protocol

1.2 Why Protocol Buffers?

Feature	Protobuf	JSON
Message Size	17 bytes (example)	65 bytes
Serialization	1.2 μ s	4.5 μ s
Type Safety	Compile-time	Runtime
Schema Evolution	Built-in	Manual
Code Generation	Automatic	Manual

Table 1: Protobuf vs JSON Comparison

1.3 Architecture

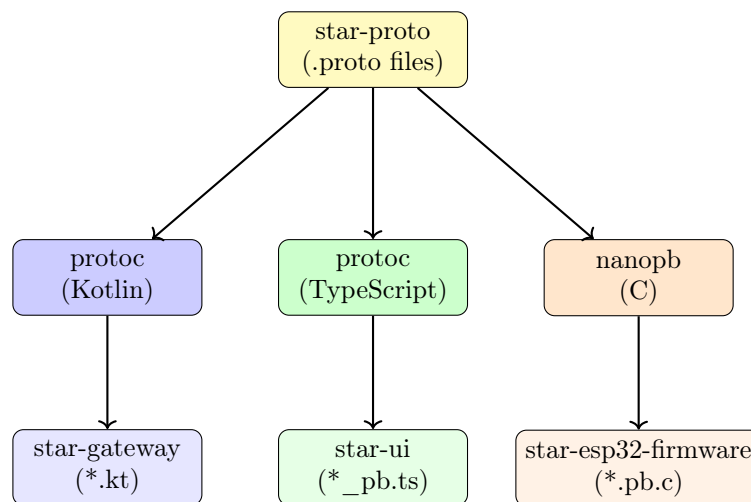


Figure 1: Code Generation Flow

2 Directory Structure

```

1 star-proto/
2 |-- README.md
3 |-- buf.yaml                                # Buf build configuration

```

```

4 |-- buf.gen.yaml                                # Code generation config
5 |
6 |-- star/
7 |   |-- motor_control.proto                    # Motor command service
8 |   |-- telemetry.proto                        # Telemetry data types
9 |   |-- navigation.proto                      # Navigation service
10 |   +-- common.proto                          # Shared types
11 |
12 |-- scripts/
13 |   |-- generate-kotlin.sh                    # Generate for star-gateway
14 |   |-- generate-typescript.sh               # Generate for star-ui
15 |   +-- generate-c.sh                        # Generate for star-esp32-firmware
16 |
17 +-- generated/                                # Generated code (gitignored)
18 |   |-- kotlin/
19 |   |-- typescript/
20 |   +-- c/

```

3 Protocol Definitions

3.1 motor_control.proto

Listing 1: star/motor_control.proto

```

1 syntax = "proto3";
2
3 package star.motor;
4
5 option java_package = "com.star.gateway.proto";
6 option java_outer_classname = "MotorControlProto";
7 option java_multiple_files = true;
8
9 import "star/common.proto";
10
11 // Motor control service for low-latency commands
12 service MotorControl {
13     // Set wheel velocities (differential drive)
14     rpc SetVelocity(VelocityCommand) returns (CommandResponse);
15
16     // Immediate emergency stop
17     rpc EmergencyStop(EmptyRequest) returns (CommandResponse);
18
19     // Set individual motor power
20     rpc SetMotorPower(MotorPowerCommand) returns (CommandResponse);
21
22     // Stream encoder data (server streaming)
23     rpc StreamEncoderData(EmptyRequest) returns (stream EncoderData);
24
25     // Bidirectional control stream (for continuous control)
26     rpc ControlStream(stream VelocityCommand) returns (stream EncoderData);
27 }

```

```
28
29 // Velocity command for differential drive
30 message VelocityCommand {
31     // Left wheel velocity in m/s (-2.0 to 2.0)
32     float left_velocity = 1;
33
34     // Right wheel velocity in m/s (-2.0 to 2.0)
35     float right_velocity = 2;
36
37     // Client timestamp in milliseconds (for latency tracking)
38     uint64 timestamp = 3;
39
40     // Optional: command sequence number
41     uint32 sequence = 4;
42 }
43
44 // Direct motor power control
45 message MotorPowerCommand {
46     // Motor index: 0=front-left, 1=front-right, 2=rear-left, 3=rear-right
47     int32 motor_id = 1;
48
49     // Power level: -1.0 (full reverse) to 1.0 (full forward)
50     float power = 2;
51 }
52
53 // Standard command response
54 message CommandResponse {
55     // True if command was executed successfully
56     bool success = 1;
57
58     // Human-readable status message
59     string message = 2;
60
61     // Round-trip latency in microseconds
62     uint64 latency_us = 3;
63
64     // Server timestamp
65     uint64 server_timestamp = 4;
66 }
67
68 // Real-time encoder feedback
69 message EncoderData {
70     // Motor index (0-3)
71     int32 motor_id = 1;
72
73     // Absolute encoder position (ticks)
74     int64 ticks = 2;
75
76     // Current velocity in m/s
77     float velocity = 3;
78
79     // Timestamp in milliseconds
80     uint64 timestamp = 4;
81 }
```

3.2 telemetry.proto

Listing 2: star/telemetry.proto

```
1 syntax = "proto3";
2
3 package star.telemetry;
4
5 option java_package = "com.star.gateway.proto";
6 option java_multiple_files = true;
7
8 import "star/common.proto";
9
10 // Telemetry streaming service
11 service TelemetryService {
12     // Get current telemetry snapshot
13     rpc GetTelemetry(EmptyRequest) returns (TelemetryData);
14
15     // Stream telemetry at specified rate
16     rpc StreamTelemetry(StreamRequest) returns (stream TelemetryData);
17
18     // Get system status
19     rpc GetSystemStatus(EmptyRequest) returns (SystemStatus);
20 }
21
22 // Stream configuration
23 message StreamRequest {
24     // Update rate in Hz (1-100)
25     int32 rate_hz = 1;
26
27     // Fields to include (empty = all)
28     repeated string fields = 2;
29 }
30
31 // Complete telemetry snapshot
32 message TelemetryData {
33     // IMU data
34     ImuData imu = 1;
35
36     // GPS data (if available)
37     GpsData gps = 2;
38
39     // Battery percentage (0-100)
40     float battery = 3;
41
42     // WiFi signal strength in dBm
43     int32 wifi_signal = 4;
44
45     // CPU usage percentage (0-100)
46     float cpu_usage = 5;
47 }
```

```
48 // Temperature in Celsius
49 float temperature = 6;
50
51 // Motor load percentage (0-100)
52 float motor_load = 7;
53
54 // Timestamp (ISO 8601)
55 string timestamp = 8;
56 }
57
58 // IMU sensor data
59 message ImuData {
60     // Orientation (degrees)
61     float pitch = 1;
62     float roll = 2;
63     float yaw = 3;
64
65     // Linear velocity (m/s)
66     float velocity = 4;
67
68     // Acceleration (m/s^2)
69     float acceleration_x = 5;
70     float acceleration_y = 6;
71     float acceleration_z = 7;
72
73     // Angular velocity (rad/s)
74     float gyro_x = 8;
75     float gyro_y = 9;
76     float gyro_z = 10;
77 }
78
79 // GPS data
80 message GpsData {
81     double latitude = 1;
82     double longitude = 2;
83     float altitude = 3;
84     float accuracy = 4;
85     int32 satellites = 5;
86 }
87
88 // System status
89 message SystemStatus {
90     ConnectionStatus connection_status = 1;
91     RobotMode mode = 2;
92     bool esp32_connected = 3;
93     bool lidar_connected = 4;
94     bool ros_connected = 5;
95     string firmware_version = 6;
96     uint64 uptime_seconds = 7;
97 }
98
99 // Connection state
100 enum ConnectionStatus {
101     CONNECTION_UNKNOWN = 0;
```

```
102 CONNECTION_CONNECTED = 1;
103 CONNECTION_DISCONNECTED = 2;
104 CONNECTION_CONNECTING = 3;
105 CONNECTION_ERROR = 4;
106 }
107
108 // Robot operating mode
109 enum RobotMode {
110     MODE_UNKNOWN = 0;
111     MODE_IDLE = 1;
112     MODE_MANUAL = 2;
113     MODE_AUTONOMOUS = 3;
114     MODE_MAPPING = 4;
115     MODE_EMERGENCY_STOP = 5;
116 }
```

3.3 navigation.proto

Listing 3: star/navigation.proto

```
1 syntax = "proto3";
2
3 package star.navigation;
4
5 option java_package = "com.star.gateway.proto";
6 option java_multiple_files = true;
7
8 import "star/common.proto";
9
10 // Navigation service
11 service NavigationService {
12     // Get current navigation state
13     rpc GetState(EmptyRequest) returns (NavigationState);
14
15     // Add waypoint to queue
16     rpc AddWaypoint(Waypoint) returns (WaypointResponse);
17
18     // Clear all waypoints
19     rpc ClearWaypoints(EmptyRequest) returns (CommandResponse);
20
21     // Start autonomous navigation
22     rpc StartNavigation(EmptyRequest) returns (CommandResponse);
23
24     // Stop navigation
25     rpc StopNavigation(EmptyRequest) returns (CommandResponse);
26
27     // Stream navigation updates
28     rpc StreamNavigation(EmptyRequest) returns (stream NavigationState);
29
30     // Stream LiDAR scans
31     rpc StreamLidar(StreamRequest) returns (stream LidarScan);
32 }
33
```

```
34 // Current navigation state
35 message NavigationState {
36     // Current robot position
37     Position current_position = 1;
38
39     // Planned path
40     repeated Position path = 2;
41
42     // Queued waypoints
43     repeated Waypoint waypoints = 3;
44
45     // Navigation active
46     bool is_navigating = 4;
47
48     // Progress percentage
49     float progress = 5;
50
51     // Estimated time remaining (seconds)
52     float eta_seconds = 6;
53 }
54
55 // 2D position with heading
56 message Position {
57     float x = 1; // meters
58     float y = 2; // meters
59     float heading = 3; // radians
60 }
61
62 // Navigation waypoint
63 message Waypoint {
64     string id = 1;
65     float x = 2;
66     float y = 3;
67     string label = 4;
68     WaypointType type = 5;
69 }
70
71 enum WaypointType {
72     WAYPOINT_NORMAL = 0;
73     WAYPOINT_STOP = 1;
74     WAYPOINT_SLOW = 2;
75 }
76
77 message WaypointResponse {
78     bool success = 1;
79     string waypoint_id = 2;
80     string message = 3;
81 }
82
83 // LiDAR scan data
84 message LidarScan {
85     repeated LidarPoint points = 1;
86     string timestamp = 2;
87     string frame_id = 3;
```

```

88 }
89
90 message LidarPoint {
91     float angle = 1;      // radians
92     float distance = 2;   // meters
93     float intensity = 3;  // 0-255
94 }
95
96 // Map data
97 message OccupancyGrid {
98     // Grid metadata
99     float resolution = 1; // meters per cell
100     uint32 width = 2;
101     uint32 height = 3;
102     Position origin = 4;
103
104     // Grid data (-1=unknown, 0=free, 100=occupied)
105     bytes data = 5;
106
107     string timestamp = 6;
108 }

```

3.4 common.proto

Listing 4: star/common.proto

```

1  syntax = "proto3";
2
3  package star;
4
5  option java_package = "com.star.gateway.proto";
6  option java_multiple_files = true;
7
8  // Empty request placeholder
9  message EmptyRequest {}
10
11 // Generic command response (shared across services)
12 message CommandResponse {
13     bool success = 1;
14     string message = 2;
15 }
16
17 // Stream request with rate
18 message StreamRequest {
19     int32 rate_hz = 1;
20 }
21
22 // Timestamp wrapper
23 message Timestamp {
24     int64 seconds = 1;
25     int32 nanos = 2;
26 }
27

```

```
28 // Vector3 for 3D data
29 message Vector3 {
30     float x = 1;
31     float y = 2;
32     float z = 3;
33 }
34
35 // Quaternion for rotation
36 message Quaternion {
37     float x = 1;
38     float y = 2;
39     float z = 3;
40     float w = 4;
41 }
```

4 Code Generation

4.1 Prerequisites

```
1 # Install protoc compiler
2 # macOS
3 brew install protobuf
4
5 # Ubuntu
6 sudo apt install protobuf-compiler
7
8 # Install Buf (recommended)
9 # macOS
10 brew install bufbuild/buf/buf
11
12 # Other platforms
13 npm install -g @bufbuild/buf
```

4.2 Using Buf (Recommended)

Create `buf.yaml`:

Listing 5: `buf.yaml`

```
1 version: v1
2 name: buf.build/star/proto
3 deps: []
4 breaking:
5   use:
6     - FILE
7 lint:
8   use:
9     - DEFAULT
```

Create `buf.gen.yaml`:

Listing 6: `buf.gen.yaml`

```

1 version: v1
2 managed:
3   enabled: true
4   java_package_prefix: com.star.gateway.proto
5 plugins:
6   # Kotlin/Java for star-gateway
7   - plugin: buf.build/grpc/java
8     out: generated/kotlin
9   - plugin: buf.build/protocolbuffers/java
10    out: generated/kotlin
11
12   # TypeScript for star-ui
13   - plugin: buf.build/community/timostamm-protobuf-ts
14     out: generated/typescript
15
16   # C for star-esp32-firmware (via nanopb)
17   - plugin: buf.build/community/nanopb-nanopb
18     out: generated/c

```

Generate code:

```

1 # Lint proto files
2 buf lint
3
4 # Generate all targets
5 buf generate
6
7 # Generate specific target
8 buf generate --path star/motor_control.proto

```

4.3 Manual Generation Scripts

scripts/generate-kotlin.sh:

Listing 7: generate-kotlin.sh

```

1 #!/bin/bash
2 set -e
3
4 PROTO_DIR="star"
5 OUT_DIR="../star-gateway/src/main/kotlin"
6
7 protoc \
8   -I=. \
9   --java_out=$OUT_DIR \
10  --grpc-java_out=$OUT_DIR \
11  --kotlin_out=$OUT_DIR \
12  --grpc_kotlin_out=$OUT_DIR \
13  $PROTO_DIR/*.proto
14
15 echo "Kotlin code generated in $OUT_DIR"

```

scripts/generate-typescript.sh:

Listing 8: generate-typescript.sh

```
1  #!/bin/bash
2  set -e
3
4  PROTO_DIR="star"
5  OUT_DIR="../star-ui/src/grpc"
6
7  protoc \
8      -I=. \
9      --js_out=import_style=commonjs:$OUT_DIR \
10     --grpc-web_out=import_style=typescript,mode=grpcweb:$OUT_DIR \
11     $PROTO_DIR/*.proto
12
13 echo "TypeScript code generated in $OUT_DIR"
```

5 Best Practices

5.1 Schema Evolution

1. **Never remove fields** - mark as **reserved**
2. **Never change field numbers** - they define wire format
3. **Add new fields** with new numbers only
4. Use **optional** for fields that may not be present

Listing 9: Schema Evolution Example

```
1  message VelocityCommand {
2      float left_velocity = 1;
3      float right_velocity = 2;
4
5      // Reserved: removed fields
6      reserved 3;
7      reserved "old_field_name";
8
9      // New optional field (added later)
10     optional float acceleration_limit = 4;
11 }
```

5.2 Naming Conventions

Element	Convention	Example
Package	lowercase.dots	star.motor
Message	PascalCase	VelocityCommand
Field	snake_case	left_velocity
Enum	SCREAMING_SNAKE	MODE_MANUAL
Service	PascalCase	MotorControl
RPC method	PascalCase	SetVelocity

Table 2: Protobuf Naming Conventions

5.3 Documentation

Always document messages and fields:

```

1  // Velocity command for differential drive robot.
2  // Sent from controller to gateway at up to 100Hz.
3  message VelocityCommand {
4      // Left wheel velocity in meters per second.
5      // Valid range: -2.0 to 2.0 m/s.
6      float left_velocity = 1;
7
8      // Right wheel velocity in meters per second.
9      // Valid range: -2.0 to 2.0 m/s.
10     float right_velocity = 2;
11 }

```

6 Integration

6.1 star-gateway Integration

Add to star-gateway/build.gradle.kts:

```

1  protobuf {
2      protoc {
3          artifact = "com.google.protobuf:protoc:3.25.2"
4      }
5      plugins {
6          id("grpc") {
7              artifact = "io.grpc:protoc-gen-grpc-java:1.60.1"
8          }
9          id("grpckt") {
10             artifact = "io.grpc:protoc-gen-grpc-kotlin:1.4.1:jdk8@jar"
11         }
12     }
13 }
14
15 sourceSets {
16     main {
17         proto {
18             srcDir("../star-proto/star")
19         }
20     }
21 }

```

```
19     }  
20   }  
21 }
```

6.2 star-ui Integration

Add npm script to `star-ui/package.json`:

```
1 {  
2   "scripts": {  
3     "proto:generate": "cd ../star-proto && ./scripts/generate-typescript.  
4       sh"  
5   }  
}
```

7 References

- Protocol Buffers Documentation: <https://protobuf.dev/>
- gRPC Documentation: <https://grpc.io/docs/>
- Buf CLI: <https://buf.build/docs/>
- gRPC-Web: <https://github.com/grpc/grpc-web>
- nanopb (C): <https://jpa.kapsi.fi/nanopb/>
- STAR Software Architecture Primer: Section 3 (Protocol Analysis)